

Scanning the Harness: An Empirical Study of Supply-Chain Defects in AI Coding-Agent Configurations

Benjamin Kapner^{1,2} Carmel Soceanu¹ Alicia Petrunin¹ Hofni Gartner¹

¹Red Hat

²Stein Faculty of Computer and Information Science, Ben-Gurion University of the Negev

Abstract

AI coding agents such as Claude Code, Cursor, GitHub Copilot, and OpenAI Codex are configured through persistent artifacts that developers write and share: instruction files, skills, commands, hooks, MCP server declarations, and subagent definitions. This *harness* is a dependency layer. It is installed from marketplaces and public repositories, runs with the developer’s privileges, and persists across sessions, yet it has no lockfile, no install-time check, and no vocabulary for what a component may do. We study this surface over 3,171 public GitHub repositories: 2,660 setups that assemble two or more component types, and 511 published skill collections. We measure only rules whose condition is decidable from bytes and whose consequence is a security exposure, a configuration that cannot work as written, or a departure from the Agent Skills specification, and we validate every finding before it counts: a second implementation of each rule re-derives it from the repository at its pinned commit (8,547 findings), a language-model adjudicator with a released prompt rules on every disagreement, and a second, independent model session outside the pipeline re-checked every counted pair against the repository and the current platform documentation; no human scoring of the verdicts has been done. Three security classes survive validation. 9.8% of setups install an MCP server with no version pinned, the pattern that vendor documentation shows; 3.1% pre-approve arbitrary command execution behind a scoped-looking grant such as `Bash(python:*)`; and 3.8% carry a skill that pre-approves the shell for whoever installs it. Together 16.0% of setups carry a security defect, 0.8% a configuration that cannot work as written, 2.4% a skill outside the specification that Claude Code nevertheless loads; 16.7% of setups carry a confirmed defect and 18.4% a confirmed finding of any class, against a raw instrument rate of 25.5% on the same rules. The first two classes appear only when components are assembled into a setup; the third ships inside published skills, in 3.7% of collections, where a marketplace scan can see it. Every rule that survives reads one file and the tree around it; the rules that compare two files re-derive at the same rate but detect differences that are usually intended, and we report them as observations. The credential exfiltration path that motivated the instrument has no confirmed instance. Two consequences the instrument stated turned out to depend on the platform version, and we report them as such. The instrument, corpus manifest, adjudication prompt, per-finding verdicts, and the independent reading are released.

CCS Concepts

- **Security and privacy** → **Software and application security**;
- **Software and its engineering** → *Software verification and validation*.

Keywords

AI coding agents, agent skills, software supply chain, configuration auditing, static analysis, harness defects

1 Introduction

Large language models have become integral to professional software development, and the tools that deliver them, Claude Code, Cursor, GitHub Copilot, Codex, and a growing set of open-source agents, are no longer interchangeable front ends to a model. Each is customized through persistent configuration: instruction files that load into every session, skills that load on demand, commands, lifecycle hooks, MCP server declarations, and subagent definitions. We call this set of artifacts the agent’s *harness*. The model may be the same across two installations; the harness is what makes one developer’s agent behave differently from another’s.

The harness is a supply chain. Its components are installed from public marketplaces and community repositories, so each installed skill is an unvetted third-party dependency whose payload is natural-language instruction and embedded scripts. This inverts the usual assumption of package management: a conventional dependency is reviewed as code, while a skill is a document that instructs a privileged agent to read credentials, run shell commands, or contact the network. The layer is no longer vendor specific, since the `AGENTS.md` format alone is read by more than twenty agents and is stewarded by the Linux Foundation’s Agentic AI Foundation [1]. The risks are documented: a large-scale analysis of two skill marketplaces reported that 26.1 percent of 31,132 skills carry a vulnerability pattern [3], a follow-up characterized the malicious subset [4], a coordinated campaign planted credential-stealing skills on a public marketplace in early 2026 [5], and CVE-2025-59536 [6] documents project-scoped configuration taking effect when a repository is opened, before the developer has approved anything.

What the harness lacks is the hygiene that package ecosystems acquired the hard way. There is no lockfile: an MCP server declared as `npm -y @scope/server` runs whatever version is published on the day the agent starts. There is no install-time check: the Agent Skills specification requires every skill to declare a name and a description. Claude Code loads a skill that declares neither and improvises both from the file, and a subagent without a description is silently never delegated to; in no case does the author hear about it. The permission vocabulary lets an entry like `Bash(python:*)` read as narrowly scoped while pre-approving arbitrary execution, and a skill can declare `allowed-tools: Bash` and carry that pre-approval into every repository that installs it. 17.5% of setups configure more than one assistant, and nothing keeps their instruction files in agreement. These are not exotic attacks. They are ordinary drift in configurations that nothing was checking.

This paper measures that drift under a standard prior prevalence claims in this area have not met. We restrict the measured rule set

to conditions that are decidable from bytes and whose consequence is a security exposure, a configuration that cannot work as written, or a departure from the Agent Skills specification. We then validate every finding of every such rule before counting it. A second implementation of the rule’s documented condition re-derives the finding from the repository re-cloned at its pinned commit. Where the two implementations disagree, or the condition holds but its consequence cannot be decided from bytes, a language-model adjudicator reads the evidence and rules on whether the stated consequence holds. A second, independent model session then re-checked every counted pair against the repository and the platform documentation, which corrected the stated consequence of two rules and a handful of individual verdicts. Across 3,171 public repositories the result is that 16.0% of the 2,660 setups carry a confirmed security defect, 0.8% a configuration that cannot work as written, 2.4% a skill outside the specification, 16.7% at least one confirmed defect, and 18.4% at least one confirmed finding of any class, against a raw instrument output of 96.8%. Two of the three security classes appear only after components are assembled; the third ships inside published skills. Every rule that survives both validation stages is decidable from one file; the rules that compare two files re-derive at the same rate and fail on consequence, because a difference between two assistants’ files is more often intended than not.

The paper contributes a validated prevalence study of harness defects across setups and collections (Sections 3 and 4); an organization of defects by the scope of analysis their detection needs, with the result that only file-level rules survive validation in this corpus (Section 3); a validation protocol that separates a rule’s predicate from its consequence and checks each with a different instrument, every verdict released (Section 3); and recommendations addressed to the party able to act on each finding (Section 5), with the instrument, corpus manifest, and pipeline from which every number regenerates.

2 Background and Related Work

Harness components. Contemporary agents expose components under different names, and we fix terminology once. *Context files* load into every session regardless of task (CLAUDE.md, AGENTS.md, copilot-instructions.md, Cursor rules). *Skills* load when the platform judges them relevant, based on a description the author writes, or, in Claude Code, on the first paragraph of the body when no description is declared. *Commands* are user-triggered workflows. *Hooks* are deterministic scripts bound to lifecycle events, and unlike skills they execute rather than persuade. *Subagents* are delegated model instances with their own tool grants. *MCP servers* are external tool providers declared in configuration and reachable at runtime. A *setup* is a concrete assembly of some subset of these for a project; a *collection* is a repository whose product is a set of publishable skills.

Agent-skill security. Liu et al. [3] conducted the first large-scale empirical analysis of the skill ecosystem, combining static analysis with model-based classification at per-file granularity, and later characterized confirmed malicious skills [4]. Schmotz et al. [7] measure agent susceptibility to skill-file attacks, and Li et al. [8] formalize a threat taxonomy validated against incidents. Practitioner

taxonomies from OWASP [31] and MITRE ATLAS [32] catalogue the attack classes; the rules measured here address configuration-side preconditions of several of them. Shen et al. [9] address the gap between what is audited and what is later loaded. Closest in spirit is the NDSS study of cross-tool harvesting and polluting in multi-tool agents [10]; automatic detection of taint-style vulnerabilities in agent frameworks [11] and red-teaming of agents through MCP tools [12] attack the same composition surface at run time. These works target marketplace artifacts, runtime behavior, or distribution integrity. We audit the installed configuration as a whole, across component types and assistant platforms, and compare the consuming population against the supplying one.

Extension ecosystems. Analyses of LLM app stores [13], ChatGPT plugins [14], and the ChatGPT app ecosystem [15] show extension ecosystems accumulating risk faster than review capacity; indirect prompt injection [16, 17] is the mechanism by which instruction-as-payload becomes execution. Wang et al. [18] frame the LLM supply chain as a research agenda; the harness is a late addition to that chain.

Configuration as an analyzable artifact. Static analysis of Infrastructure as Code [19], catalogs of configuration smells [20], and security smells in Puppet [21] established that configuration files benefit from structural validation and cross-file consistency checking. That tradition has reached agent configuration: a catalog of smells in AGENTS.md files [22], an empirical study of 2,303 context files [23], a controlled evaluation finding that context files often raise inference cost without raising task success [24], and a measurement of their effect on agent efficiency [25]. All operate on the context file alone; AutoSaddler [26] treats the harness as code and optimizes it from execution traces.

Precision of static analyzers. Static analyzers are known to overwhelm users with warnings whose majority are irrelevant or wrong [27, 28]; ranking and pruning techniques exist because raw output cannot be taken at face value [29], and empirical comparisons report precision varying by an order of magnitude across tools [30]. We adopt the standard this literature has long argued for and extend it: every reported finding is re-derived by an independent implementation against the repository that produced it, and every disagreement between the two implementations is adjudicated before a finding counts.

3 Study Design

We ask three questions. **RQ1:** which security and configuration defects can we confirm harness repositories carry, and how often? **RQ2:** how do assembled setups differ from the collections that supply them? **RQ3:** which confirmed classes are attributable to a missing convention in the formats or platforms, and what convention would remove them?

Instrument. harness-eval is an open-source static analysis tool, distributed as a Python package, that discovers harness components across Claude Code, Cursor, Copilot, Gemini CLI, OpenCode, Windsurf, Cline, and Codex layouts, parses each into a typed representation, and reports terminal, JSON, or SARIF output. A pluggable engine runs deterministic rules, each emitting a structured finding with severity, location, and remediation. No rule invokes a model: deterministic rules run in milliseconds, return identical results

Table 1: The 31 measured rules by required analysis scope.

Scope	#	Examples
FILE	20	unpinned MCP package, arbitrary-execution grant, prompt-mode bypass, skill tool pre-approval, frontmatter that does not parse
FILE_FS	7	missing hook script, committed per-machine settings, broken <code>@import</code> , symlink escaping the tree
PAIRWISE	4	agent declaring an absent skill, context files or MCP declarations that diverge across assistants

on identical input, and operate offline, which is what allows the check to gate a build. This study measures 31 of its rules, selected by the criteria below; the instrument ships further heuristic rules that judge prose, and they play no part in any figure. Setup-scope analyses run over a component graph whose edges are explicit invocations among components; edge extraction distinguishes an invocation addressed to the agent from a mention or quoted display text, because an edge inferred from prose manufactures exactly the flows the analyzer exists to find. We classified each measured rule by scope by reading what it consumes; an automated pass was reviewed by hand with an override table, and both verdicts are released.

Analysis scope. We organize defects by the scope a detector must read (Table 1), a property decidable from the rule’s implementation. *FILE*: the bytes of one component (malformed frontmatter, an over-broad grant, a skill that pre-approves the shell). *FILE_FS*: the file plus the surrounding filesystem (a missing hook script, a committed per-machine settings file). *PAIRWISE*: two components compared (diverged context files or MCP declarations, an agent declaring an absent skill). *SETUP*: the whole component graph (a credential-reading component with a delegation edge to a network-capable one, an exfiltration path neither file exhibits alone). A per-file marketplace scanner is restricted by construction to *FILE* and part of *FILE_FS*. The taxonomy states what a detector must read, not what current detection delivers: in this corpus no *PAIRWISE* or *SETUP* rule survives validation (Sections 4.5 and 4.7), so the results measure the *FILE* and *FILE_FS* levels and bound the levels above.

Corpus. The corpus is drawn from the supply side of the ecosystem by design: the public lists and plugin marketplaces through which configurations are recommended and installed, since those are the configurations that propagate. We analyzed 3,171 public GitHub repositories, discovered through assistant-specific topic queries (claude-code, agent-skills, agents-md, cursor-rules, copilot-instructions, opencode, subagents, and others), README references to canonical context filenames, thirty public curated lists of agent configurations, and the plugin marketplaces of Claude Code and its community. The discovery channel is recorded per repository. Forks and archived repositories were excluded. For each repository the pipeline performed a shallow clone, recorded the released commit, ran the analyses under a timeout, recorded findings and inventory metadata, and deleted the working copy. Of 9,295

candidate repositories scanned, 3,065 carried no harness component, 1,633 carried only an instruction file, 1,322 carried a single component type with fewer than five skills, 65 were forks, template-derived copies, or renames of another entry, and 22 failed to clone or scan. The remainder was rescanned from scratch on the released instrument version, which dropped 17 more (a clone that failed on the second pass, or a fork that the corrected discovery revealed), leaving the 3,171 that form the corpus defined below. No third-party content is retained; the released artifact consists of URLs, pinned commit identifiers, and findings.

Strata. Repositories are classified from the tool’s own inventory into two strata, reported separately and never pooled. A *setup* assembles at least two distinct component types: a context file and skills, skills and hooks, an MCP declaration and settings. A *collection* has five or more skills and no component that composes them. This yields 2,660 setups and 511 collections. Four kinds of discovered repository are outside the corpus: those in which the tool found no harness component (README mentions, tutorials, lists; 3,065), those whose only component is an instruction file such as `CLAUDE.md` or `AGENTS.md` (1,633; no measured rule reads those files), those with a single component type and fewer than five skills, such as one `.mcp.json` or one skill (1,322; a file, not a configuration), and those whose clone or scan failed. A fork or a template-derived copy that carries the same component inventory as another entry, or a rename that resolves to the same pinned commit, is counted once (65). Collections and setups are the two sides of the supply chain, what developers install and what they end up running.

The measured rule set. A rule carries a figure only if it meets three conditions. Its predicate is decidable from a file, the filesystem, or a comparison of two files, never from a judgment about prose. Its consequence is a security exposure (family S), a configuration that cannot work as written (family Q), a departure from the Agent Skills specification that the reference client tolerates (family P), or an inconsistency between two assistants’ copies of one declaration (family C). And it passed the validation stages below. 15 rules are in family S, 12 in family Q, 2 in family P, and 2 in family C.

Consequences are dated. What a condition means depends on the client version that reads it, and the clients move. We state every consequence against the platform documentation current at the time of writing (September 2026) and record where it changed. Two cases matter for the figures. The Agent Skills specification requires a frontmatter block with name and description; the current Claude Code documentation makes every field optional, defaults the name to the directory and the description to the first paragraph of the body, and loads a file with no block at all as skill text. A skill without either is therefore a specification violation that one consumer tolerates and another may reject, which is family P, not a skill that cannot load. And `permissions.defaultMode: bypassPermissions` in project-scoped settings was honoured by Claude Code until v2.1.257 and is ignored from project and local settings since; the condition is reported with that note. `enableAllProjectMcpServers`, which the instrument reported under the same rule, pre-approves the project’s MCP servers and leaves tool permission prompts in place; it is split off as an advisory class and carries no figure.

Validation, stage one: independent re-derivation. The pipeline re-derived every finding of every measured rule, not a sample. It re-cloned each flagged repository at its pinned commit and re-derived each finding with a second implementation of the rule’s documented condition, written without reference to the instrument’s code: it discovers components, parses configuration, and evaluates the predicate on its own. In total it re-derived 8,547 findings across 1,115 re-cloned repositories. The stage measures agreement between two implementations of the same condition, which bounds the instrument’s implementation error: a finding both produce from the same bytes is a property of the repository. The second implementation also sub-classifies each re-derived finding (a skill with no frontmatter block against one whose block lacks a field; an npx package that is absent from the project’s own manifest against one it declares), which the next stage consumes.

Validation, stage two: adjudication of disagreements. The unit of validation is the (repository, rule) pair, since prevalence counts repositories. A pair is *agreed* when the second implementation re-derives every finding in it and at least one carries the stated consequence, and an agreed pair counts as a defect without further review. A pair is a *disagreement* when any finding was refuted or could not be re-derived, when every finding falls in a sub-class whose consequence does not hold (a test fixture, a template the repository ships for others, an npx package the project’s own lock-file pins), or when the rule compares two files or resolves a reference and the second implementation can establish the difference but not whether it is intended. For every disagreement the pipeline extracts the evidence a reviewer would look at, the declaration and the lines around it, the file’s location in the tree, and for a pairwise rule both sides of the comparison, and submits it with the rule’s definition to a language-model adjudicator (Claude, through the Anthropic API) that returns one of *defect*, *not a defect*, or *uncertain*, a reason code from a fixed list, and a plain-language explanation of what the file contains and why that is or is not a problem for someone who clones the repository. Only a verdict of *defect* lets a disagreement count. The prompt, the reason codes, and every verdict and explanation ship in the artifact. Across the validated rules, 744 pairs were re-derived, 586 agreed, 158 went to the adjudicator, and 640 pairs count as defects in the figures below.

Validation, stage three: an independent model re-reads every counted pair. The two stages above check the predicate and, for disagreements, the consequence; an agreed pair counts without anyone reading it. As a check on that, a second model session, separate from the adjudicator, with web access, given the table of every (repository, rule) pair that the first two stages had produced (743 rows, each with the harness file, its URL at the pinned commit, and the instrument’s message) and no access to the audit’s or the adjudicator’s columns, re-examined every pair at its pinned commit and recorded whether it is a defect and a one-line reason. This is a model, not a human reviewer; no human scoring of the verdicts has been done, and Section 6 says what that leaves open. The review is released as a filled copy of the table. Where it objected at the level of a rule, the objection was checked against the platform documentation and, where it held, changed the instrument and the families above; where it objected to an individual pair, the pair was re-examined at its pinned commit and the verdict corrected where

the objection held. The corpus was then rescanned on the corrected instrument, and every figure in this paper comes from that rescan.

In plain terms, the study is four passes over the same repositories: the scanner reports, an independent script re-derives, a model reads the evidence where the two disagree, and a second model re-reads every pair that survives. Each pass exists because the one before it can be wrong in a way the next can see; Section 6 reports where the two models disagreed and, on re-examination, which was right.

Tiers. A rule is *gating* when the two implementations agree on at least 97 percent of at least fifty re-derived findings and at least 80 percent of its re-derived pairs end as defects; 6 rules meet that bar and every headline figure counts only them. A rule is *provisional* when it agrees without exception on at least thirteen but fewer than fifty findings and passes the consequence bar; 1 rule is in that state, marked ^P in every table, and enters no headline union. A rule is an *observation* when it agrees at the bar and fewer than 80 percent of its pairs end as defects: the condition is real and mechanically re-derivable, and what it detects is more often intended than not. 6 rules are in that state (Section 4.5); they carry no prevalence figure and enter no union. A rule with fewer than thirteen findings in the corpus carries no figure. The accounting over the 31 measured rules: 6 gating, 1 provisional, 6 observations, 1 below the agreement bar, 9 with too few findings, and 8 with no finding in the corpus; the artifact’s rule catalog lists every one with its outcome. For a gating or provisional rule, a repository counts toward prevalence only if its pair is agreed or adjudicated a defect. Per-rule agreement with Wilson intervals, pair counts, and adjudication outcomes are in Table 5. We validated the rules on the released tool version and record its commit in every result. The selection effect of the bars remains: rules whose consequence follows from bytes alone are the ones that pass, so the gating set is file-level and every headline figure is a lower bound on the rules we have, with recall unmeasured (Section 6).

Reference resolution is advisory. The highest-volume error rule reports that a path referenced in a skill body does not resolve. Its literal claim re-derives at 100 percent on 112 sampled findings, but extracting a path from prose is a judgment about text and the claim decomposes by consequence, so the rule and its command-file counterpart are reported outside every union (Section 4.7).

4 Findings

4.1 What Harnesses Contain

The median setup carries 6 components and 7,057 tokens, and the largest carries 500 components and 1,611,649 tokens. Median always-loaded content is 1,198 tokens in setups, consumed on every session regardless of task, against 189 in collections. Multi-assistant configuration is current practice: 17.5% of setups configure more than one assistant, counting only assistants detected from a tool-specific file (AGENTS.md is read by OpenCode, Codex, and others and attributes to none of them). The assistants detected, with repository counts, are Claude Code (1887), Copilot (324), Cursor (315), Gemini CLI (273), Codex CLI (212), OpenCode (142). A tool restricted to one vendor’s layout would miss part of the configuration in 17.5% of the setups it examined.

16.0% of setups (95% CI 14.6 to 17.4) carry a confirmed security defect from the gating rules. 0.8% carry a configuration that cannot

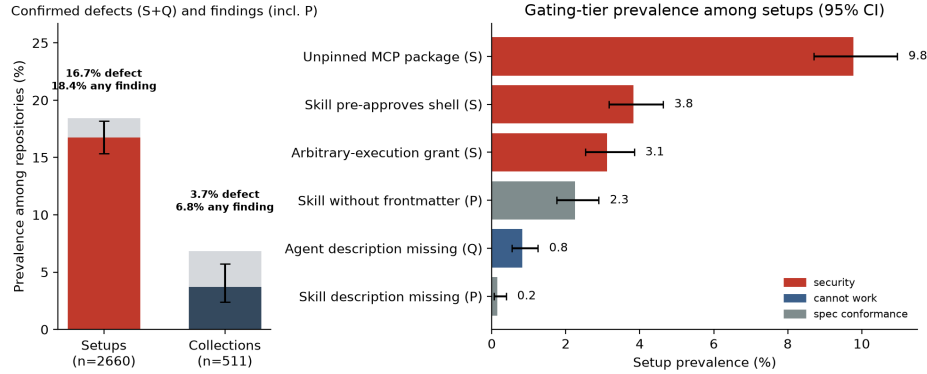


Figure 1: Prevalence over the 6 gating rules. Left: setups and published collections with any confirmed defect (security or cannot-work, solid, with 95% Wilson interval) and with any confirmed finding once the specification family is included (light). Right: per-rule prevalence among setups, coloured by family, with 95% Wilson intervals.

Table 2: Confirmed defects by rule and stratum, over 2,660 setups and 511 collections. Family: S security, Q cannot work as written, P specification conformance, C cross-assistant. ^P provisional. The reference rule is reported apart because its findings decompose by consequence.

Rule (family)	Scope	Setups	Collections
Unpinned MCP package (S)	FILE	9.8%	0.0%
Skill pre-approves shell (S)	FILE	3.8%	3.7%
Arbitrary-execution grant (S)	FILE	3.1%	0.0%
Agent description missing (Q)	FILE	0.8%	0.0%
Skill without frontmatter (P)	FILE	2.3%	3.5%
Skill description missing (P)	FILE	0.2%	0.0%
Local settings committed (S) ^P	FILE_FS	0.7%	0.0%
Any security defect (S, gating)		16.0%	3.7%
Any cannot-work defect (Q, gating)		0.8%	0.0%
Any spec-conformance defect (P, gating)		2.4%	3.5%
Any confirmed defect (S and Q, gating)		16.7%	3.7%
Any confirmed finding (S, Q, and P, gating)		18.4%	6.8%
Any confirmed finding (incl. provisional)		18.8%	6.8%
Raw gate output before audit		25.5%	9.6%
Reference does not resolve	FILE_FS	33.1%	64.0%

work as written, and 2.4% a skill outside the Agent Skills specification. Counting the security and cannot-work families together, 16.7% of setups (95% CI 15.4 to 18.2) and 3.7% of collections carry at least one confirmed defect; counting all 6 gating rules, the specification family included, 18.4% of setups (95% CI 17.0 to 19.9) and 6.8% of collections carry at least one confirmed finding (Figure 1, Table 2; an anonymized instance of each class is in the artifact). Throughout, “confirmed” means the second implementation re-derived the rule’s predicate from the repository and the pair is either agreed or adjudicated a defect.

The raw rate of the measured rules before validation is 25.5% of setups against 18.4% confirmed; most of the gap is the observation tier, rules whose condition re-derives in every case and whose consequence the adjudicator rejects in most (Section 4.5). Prior work [3, 7] reports prevalence per skill from unaudited detector output, of the same order and subject to the same inflation.

4.2 Security: Nothing Is Pinned, Grants Read as Scoped, Skills Pre-Approve the Shell

9.8% of setups declare at least one MCP server whose package is unpinned: the typical form is `npx -y @scope/server-name` with no version, and `uvx` and `docker` without a tag or digest appear in the same role. The server runs with the agent’s privileges, is fetched from a public registry on each start, and executes whatever the registry serves that day. This is the dependency hygiene problem that `npm`, `PyPI`, and `Docker` each solved with lockfiles and digests, and the MCP configuration formats have no equivalent. Collections carry 0.0%: they ship skills, not server declarations, so the risk enters at assembly time. Of the 272 repositories whose declaration was re-derived, 258 were agreed and 14 went to the adjudicator, which confirmed 2; the rest are test fixtures, templates the repository ships for other projects, or packages the project’s own manifest and lockfile pin, and they are not counted. The smell is reachable: in a seeded sample of 40 setups with an unpinned server, re-cloned at their pinned commits, 29 ship a skill, command, or context file that names the server or one of its tools.

The three security classes are not one severity, and we report them apart so a reader can weigh them. An unpinned server is the pattern that the MCP vendor documentation shows, and the exposure resembles an unpinned build dependency, with one difference: the package resolves on every session start and runs with the agent’s privileges rather than once at build time. A committed prompt bypass or a grant that pre-approves an interpreter removes a control the user believes is in place. We count all three because each lets code run without the user’s decision, and the per-class figures let a reader who ranks them differently recombine them.

3.1% of setups pre-approve arbitrary command execution through a `permissions.allow` entry that reads as narrowly scoped: `Bash(awk:*)`, `Bash(python:*)`, `Bash(find:*)`, `Bash(sed:*)`. `awk` evaluates commands via `system()`, `python -c` runs anything, `find -exec` spawns processes, and `GNU sed` runs commands through its `e` flag. The security class is an explicit, published list: shells and shell wrappers, interpreters, package runners (`npm`, `bunx`, `uvx`, `pipx`), tools with a documented shell escape (`awk`, `sed`, `find`), plus

Bash(*) and a bare Bash. Among the setups counted (83 setups carry 161 grant findings; each setup counts once per mechanism, so the mechanisms sum to 159), 19 grant an unrestricted shell, 54 an interpreter, 18 a package runner, 58 a shell-escape tool, and 10 a shell by name. Grants on curl and wget (0.9%), on make, docker, ssh, and editors (0.6%), and bare Edit or Write (1.3%) are reported as advisory classes outside the security count: a project that grants make usually means it. The rule is silent on Bash(git:*). The grant is exercised: of the 65 setups whose grant is in the security class, all re-cloned at their pinned commits, 22 ship a component that invokes the granted interpreter or tool in an instruction context. That cuts both ways. It shows the grant is live, and it shows that many authors granted the interpreter because they wanted it run. Whether they understood that Bash(python:*) is Bash(*) is a claim about expectations that no static audit can measure.

The third class travels with the skill. A skill’s frontmatter may declare allowed-tools, the tools the client runs without a prompt while the skill is active. 3.8% of setups and 3.7% of collections carry a skill whose list includes an unrestricted Bash or a shell-escape command, so installing the skill installs a shell pre-approval. The second implementation re-derived every one of 898 entries; only entries that pre-approve arbitrary execution are counted, and entries scoped to a specific command or to file writes are not. Of 121 repositories re-derived, 121 count. This is the one security class that appears at publication time rather than assembly time, and it is the one a marketplace scan can see.

One neighbouring class is provisional, confirmed without a false positive on too few findings to gate: 0.7% of setups commit .claude/settings.local.json, a per-machine file the client gitignores on creation, shipping one developer’s approved grants to every clone. The committed prompt bypass is rarer than the first version of this study reported. 0.4% of setups commit permissions.defaultMode of bypassPermissions or dontAsk in project settings, every one re-derived, which removed the permission prompt for every user who opened the repository on Claude Code before v2.1.257 and is ignored from that scope since; the setting is still committed and still takes effect on older clients, and with 12 findings it is below the count that carries a figure. The instrument had reported enableAllProjectMcpServers under the same rule; that setting, committed by a further 0.8% of setups, pre-approves the project’s MCP servers, including ones not yet added, and leaves tool permission prompts in place. It is an advisory class and enters no figure. Committed life-cycle hooks, present in 6.8% of setups, are reported as an execution surface rather than a defect: current clients prompt before running project hooks, and the hooks’ commands are where a defect would be. Hooks that force-push or pipe a download into a shell, literal API keys in MCP declarations, credential files committed inside a skill, and symlinks escaping the tree each occur in a handful of setups, too few to carry a figure; they are listed in the artifact.

4.3 Configuration That Cannot Work as Written

A second group of conditions shares one property: the configuration is syntactically present and cannot do what it says. Together they touch 0.8% of setups and 0.0% of collections. Ordered by prevalence among setups: agent description missing in 0.8% (22).

Table 3: Rules that re-derive mechanically and fail the consequence bar. Pairs is the number of (repository, rule) pairs re-derived; Adj. the disagreements the adjudicator read; Defect the pairs that count, with the share of all pairs; and the most common reason code among the rest.

Rule (family)	Pairs	Adj.	Defect	Most common non-defect reason
Broken @import in context file (Q)	23	23	11 (48%)	runtime created file
Agent declares absent skill (Q)	7	7	3 (43%)	external dependency
Context files diverge across assistants (C)	71	71	29 (41%)	per assistant variant
MCP server diverges across assistants (C)	22	22	8 (36%)	equivalent declarations
MCP endpoint integrity (S)	5	4	1 (20%)	other
MCP config structurally invalid (Q)	13	13	1 (8%)	shipped template

0.8% of setups define a subagent with no description. The Claude Code documentation is explicit that such a file is skipped and the reason written only to a debug log, so the subagent exists and is never delegated to; 22 of 23 re-derived repositories count. It is a one-line check that the client performs and does not report. Three further conditions in this family, a subagent that declares an absent skill, a context file whose @path import points at nothing, and an MCP configuration with no reachable server, re-derive mechanically and fail the consequence bar; they are reported in Section 4.5.

4.4 Skills Outside the Specification

The third family is the one whose consequence we had to correct. The Agent Skills specification requires every SKILL.md to open with a frontmatter block that declares name and description. 2.3% of setups and 3.5% of collections carry a skill with no block at all, and 0.2% of setups a skill whose block lacks the description; together 2.4% of setups and 3.5% of collections (skill without frontmatter in 2.3% (60); skill description missing in 0.2% (4)). The instrument, and the first version of this study, stated the consequence as “the skill never loads.” The independent reading (Section 6) objected, and the current Claude Code documentation bears it out: every frontmatter field is optional, a missing description falls back to the first paragraph of the body, and a file without a block is loaded as skill text. In Claude Code such a skill loads and is selected on an improvised description; a consumer that validates against the specification, as its reference validator does, rejects it. What the family measures is therefore not a broken skill but a specification the reference client does not enforce and authors do not follow, at a rate that a marketplace scan can see at publication time. That the specification and the implementations disagree about what a valid skill is, and that authors receive no signal either way, is the finding for the format’s stewards. A skill whose block is present but lacks only the name is reported under the same rule and kept out of every count as an advisory class, since every client defaults the name to the directory.

4.5 Observations Below the Consequence Bar

Table 3 lists the 6 rules on which the two implementations agree at the bar and fewer than 80 percent of pairs end as defects. Each detects a condition that holds; the adjudication shows the condition is more often intended than not, so a client that reported it would be reporting a review candidate. The pairwise rules fail on intent: the second implementation can establish that two assistants’ context files or MCP declarations differ and cannot establish that the

difference is a mistake, and the adjudicator read most differences as deliberately assistant-specific or equivalent under each client’s syntax (29 of 71 drift pairs and 8 of 22 divergence pairs count). The cross-file rules fail because the file the reference names exists somewhere the check cannot see: a per-machine file the client creates at run time, a template the repository ships for other projects, a skill in another installed plugin, a placeholder in an official bundle, a host inside a private network. Every one of these is a one-line check a client could run at load time, and every one needs a convention the format lacks before it can be reported without a reader: a declared per-machine import, a declared template directory, a declared plugin dependency, a declared internal network.

4.6 Setups Against Collections

The two strata differ in composition, not only in rate (Table 2). Unpinned servers and arbitrary-execution grants occur only in setups, because a collection ships skills and none of those files. Shell pre-approval runs the other way: it is a property of the skill’s own frontmatter, so it travels from the collection into every setup that installs the skill, and collections carry it at a higher rate than setups. The supply-chain reading is therefore two checks, not one. Installation-time auditing of a collection catches unparseable frontmatter and shell pre-approval; a second check at assembly time is still required, because the server and grant classes appear only after assembly. Among setups, the 2,100 discovered through public lists that the community maintains as recommendations carry confirmed defects at 18.9%, against 18.4% for setups overall. Recommendation is not review.

4.7 A Validated Negative Result and the Reference Split

The class of defect that motivated the instrument is a component with credential or environment access holding a delegation edge to a network-capable component. With edges restricted to explicit invocations, the rule fired on 6 repositories in the corpus, each re-cloned and read (the reading is in the artifact). In one the credential read was a token-counting routine that happened to match the pattern. In the other four an API key read from the environment or a project .env was sent to the vendor API that issued it, by the same author’s components, which is the intended use of the key and not a crossing of any trust boundary; in none was the delegation edge a hand-over of data rather than an instruction to run the second skill. No repository in the corpus exhibits a credential-to-network exfiltration path. We report the class as absent, and note what the search taught: a component graph is only as sound as the extraction that builds its edges, and a mention in prose must at most contribute a low-confidence signal, never a security finding on its own.

The reference rule fires on 33.1% of setups and 64.0% of collections, the highest raw rate of any error-severity rule, and it is right about its literal claim: read against consequence, its findings split roughly 45 percent dead paths (an agent that follows the instruction fails), 20 percent real files named by the wrong path, and 40 percent files the skill will *create* in the consuming project (“write the plan to plans/current.md”), which are correct and will never resolve statically. The planned-output class is concentrated in collections and is invisible to any resolution check because nothing in the skill

Table 4: Findings, the party able to act, and the recommendation.

Finding	Owner	Recommendation
Unpinned MCP servers (9.8%)	MCP, clients	A lockfile for server declarations; clients refuse or warn on unpinned npx/uvx/docker
Scoped-looking grants (3.1%)	Claude Code	Interpreter-aware permission UI: show Bash(python:*) as “any command”
Skills that pre-approve the shell (3.7% of collections)	Marketplaces, Skills spec	Show allowed-tools at install time; require a scoped form
Prompt bypass committed (0.4%, below the figure bar)	Clients	Refuse defaultMode bypass in project-scoped settings, as Claude Code does since v2.1.257
Skills outside the spec (2.4%)	Skills spec, clients	Enforce the required fields at load time and say so, or drop them from the spec
Planned-output references (Sec. 4.7)	Skills spec, AAF	A creates: frontmatter field; tooling skips declared outputs
Context-file drift (observation)	AAF, clients	One source of truth; clients honour @AGENTS.md imports
Raw gate output exceeds confirmed (25.5% vs 18.4%)	Tool authors	Re-derive findings from the corpus before publishing a rate

formats lets an author say which paths are inputs and which are outputs. The coding was automated from the sentence around each reference and its precision is not measured.

5 Implications and Recommendations

Each finding above is attributable to a missing convention, and each convention has an owner. Table 4 summarizes.

For format stewards. The single most consequential gap is that a skill cannot declare what it creates. A creates: list in skill frontmatter would let tooling separate a dead reference from a planned output mechanically and would tell a user at install time which files a skill will write into their repository. The allowed-tools field needs the same attention in the other direction: it is the one place a published skill can carry an execution pre-approval, and the specification should require a scoped form or a marketplace should display the field. For the AGENTS.md format, the drift finding argues for making one file the source of truth and having every client honour an import rather than a copy. For the Agent Skills specification, the frontmatter findings show that a validity rule the reference client does not enforce is a rule authors do not follow: 3.5% of published collections ship a skill the specification’s own validator rejects and Claude Code loads without comment. Either the fields are required, and the clients should say so at load time, or they are not, and the specification should say that.

For MCP and the clients. Server declarations need the equivalent of a lockfile. The minimum is a client that warns when npx -y carries no version; the right answer is a resolved manifest with digests, generated on first run and committed beside the configuration, as every package manager eventually did. Permission vocabularies need to say what an entry means: Bash(python:*) should render as “any command” in the interface that asks a user to approve it. defaultMode: bypassPermissions should not be honoured from project-scoped settings at all, because a project setting is written by one person and executed by everyone who clones;

Claude Code adopted exactly this in v2.1.257, during the study, and the committed settings we count remain live on every earlier client and on any other client that reads the file.

For marketplaces and teams. Collections and setups fail differently, so two checks are needed. A marketplace should run the frontmatter and allowed-tools checks on a collection at publication time. A team should run the configuration-level checks at the pull that introduces a component, where unpinned servers, arbitrary-execution grants, and diverged declarations live; the released instrument runs in seconds and emits SARIF, so it attaches to code scanning like any dependency check.

What static analysis can and cannot establish. Every rule in this study is decidable from the bytes of a configuration, which lets the check run in a CI gate in milliseconds with no model and no execution, and which bounds what it can say: that a skill violates its specification, a server is unpinned, or a grant permits arbitrary execution, not whether the skill improves the agent’s work, the server behaves, or the grant is exercised harmfully. Kevin et al. [2] make the complementary case: their ACES framework evaluates skills by paired live trials with and without the skill, and across 145 skills and 947 paired cases structural scans and measured effect correlate at 0.14. The static gate is the health check and the install-time guard; evaluating what a component does requires executing it in a sandbox against representative tasks with a model judging the trace, and the fields that would let a static tool hand off to such an evaluation (a declared output list, a tool budget, a test task) are the conventions the formats lack.

For tool authors and for studies built on tools. The gap between 25.5% and 18.4% generalizes beyond this instrument. A predicate that re-derives in every case can still name a defect in a minority of them, and the difference is invisible from the detector’s output. Re-deriving every finding is a few hours of compute on a corpus of this size, and reading the evidence behind each is a few hours more with a model in the loop. A prevalence figure taken from unaudited detector output is a measurement of the detector.

6 Threats to Validity

Discovery is list-, marketplace-, topic-, and README-based rather than path-based, so the corpus over-represents repositories that advertise or distribute their agent tooling. That is the population this study targets, the configurations that get recommended and installed, and absolute rates describe it; a path-based corpus would add repositories that keep a harness for private use, and we would expect the unpinned-server rate to rise there, since every repository with an `.mcp.json` would enter (the per-channel rates are in the artifact and differ little). Validation measures agreement on each rule’s predicate and adjudicates the disagreements; it does not measure recall. We do not know what the 6 gating rules miss, and every headline figure is a lower bound on the defects the instrument can express. The second implementation was written by the same team as the study, against the rules’ documentation rather than their code; a shared misreading of a format would pass both. An agreed pair counts without a reader, so a defect both implementations compute from the same bytes and a developer would still not fix is counted; the sub-classes that route fixtures, templates, and locally

pinned packages to adjudication are the guard, and they are as complete as the cases we knew to write. **Both models make mistakes, and no human has scored them.** The adjudication verdicts come from one language model under one prompt, and the stage-three review is a second model session, not a person; neither is an oracle, and the study is designed so that each can catch the other, with the authors re-examining the pairs on which they disagree. What remains undone is a human score of the adjudicated pairs against the evidence; the artifact ships the table and a scoring script for it. Before its corrections were applied, the review agreed with the pipeline’s decision on 92.2 percent of the pairs then in the table (Cohen’s $\kappa = 0.72$); against the final table it agrees on 93.3% of 743 pairs ($\kappa = 0.76$): 99.3% of the 586 pairs both implementations had agreed on, and 70.7% of the 157 pairs the model had adjudicated ($\kappa = 0.23$). Most of the remaining disagreement is on the two intent rules that carry no figure, where the review treats a textual difference between two assistants’ files as unproven unless the copies contradict each other and the adjudicator counted any substantive section difference. On the adjudicated pairs that touch a headline figure, the 18 pairs of gating and provisional rules, the review agrees on 10 of 18; every disagreement is an unpinned package inside a shipped template, an example directory, or a test fixture, on 6 of which the review says defect where the study excludes the pair and on 2 the reverse, and it never disagrees on a live unpinned server. Whether a template counts is a policy the paper resolves by sub-class; the review itself reversed on it between its two passes, and either choice moves the headline by at most 0.3 points. The rest we re-examined one by one at the pinned commit. The adjudicator had been wrong nine times in ways a reader could see: it accepted a pinned package the instrument had misread (uvx -python with the spec after the flag), two servers marked disabled, a package declared in the project’s own manifest, skills in a git submodule, a Copilot instructions file discovered as a subagent, a Cursor file mention read as an import, a file the project’s setup command creates, and it had excluded a genuine broken import as a fixture. The reviewing model had been wrong too: an earlier pass rejected every skill-frontmatter pair under the instrument’s old “never loads” wording and accepted them all once it was corrected, and it reversed itself on whether a template counts. One objection did not hold: a reference document stored as `.github/agents/*.agent.md`, which Copilot loads as a custom agent and requires a description for. Five of the adjudicator’s errors were the instrument’s first and are fixes in the released version; the rest are corrected verdicts, shipped with the reviewed table. The mechanical re-derivation that precedes both readers is deterministic and exhaustive, which bounds the role of either to the disagreements (158 of 744 pairs) and to the final pass, and the prompt, the reason codes, and every verdict ship in the artifact so that a reader can re-run the model stage with another model or read the evidence directly. Consequences are stated against the platform documentation of September 2026; a client that changes what it does with a field changes what the corresponding rate means, as `bypassPermissions` did during this study, and the rates should be read with the version note attached to each rule. The 80 percent bar is a choice; the closest rule to it, subagent description missing at 95.7%, would move to the observation tier at a 90 percent bar without changing any security figure. Stratum assignment derives from the tool’s own inventory, so a component type it does not

recognize would misplace a repository; the manifest records the inventory behind every label. Finally, the corpus is public GitHub; configurations maintained privately inside organizations may differ systematically and are the population most relevant to enterprise adoption.

7 Conclusion

The AI code-agent harness is a supply chain surface that arrived without the hygiene other dependency layers acquired: no pinning, no install-time validation, no way to declare outputs, no single source of truth across assistants, and a skill format that lets a published component carry a shell pre-approval into every repository that installs it. Measured over 3,171 repositories and counting only findings re-derived at pinned commits and read against their consequence, 16.0% of setups carry a confirmed security defect, 0.8% a configuration that cannot work as written, 2.4% a skill outside the specification, and 16.7% a confirmed defect of either kind, and 18.4% a confirmed finding once the specification family is included. The defects are ordinary, they fix with a local edit, and most trace to a convention the formats or clients could adopt. The rules that survive validation are the ones whose consequence follows from bytes; the rules that compare two files detect real differences that are usually intended, and they need a convention before a client can report them. A rate reported from raw analyzer output is a property of the analyzer until the findings have been re-derived from the repositories that produced them and read against what they mean.

Data Availability

The instrument is available at <https://github.com/redhat-community-ai-tools/harness-eval> and the study artifact, comprising the rule catalog with its scope classification, the corpus manifest listing every analyzed repository with its pinned commit, all findings, the validation pipeline with the re-derivation verdicts, the adjudication prompt and per-finding verdicts, and the scripts that regenerate every number in this paper, at <https://github.com/Benkapner/harness-eval-experiments>. No third-party repository content is redistributed.

Ethical Considerations

All analyzed repositories are public and were evaluated in read-only mode; no exploit was developed or run. Every finding concerns configuration already publicly visible in the repository that contains it. The body refers to repositories by class; the released artifact necessarily lists URLs, pinned commits, and findings per repository, so the paper claims no more anonymity than public repositories have. Validation found no confirmed instance of cross-component credential flow, so no coordinated disclosure was warranted; the released manifest lets a maintainer locate their own entry. The few repositories committing a literal API key are identified in the artifact by pinned commit only and their values are not reproduced anywhere. The study involved no human subjects and no personally identifiable information.

A Validation Table

Table 5 gives, for each rule that carries a prevalence figure, the agreement between the instrument and the second implementation, the pairs re-derived, and the outcome of adjudicating the disagreements; the same table over all 31 measured rules is in the artifact.

Table 5: Validation of the reported rules. Audited is the number of findings the second implementation re-derived; Agreement the share it confirmed, with a 95 percent Wilson interval; Pairs the (repository, rule) pairs re-derived; Adjud. the disagreements the adjudicator read; Defect the pairs that count toward prevalence (agreed, or adjudicated a defect), with the share of all pairs.

Rule (family)	Audited	Agreement (95% CI)	Pairs	Adjud.	Defect
Unpinned MCP package (S)	961	100.0% (99.6–100.0)	272	14	260 (96%)
Skill pre-approves shell (S)	898	100.0% (99.6–100.0)	121	0	121 (100%)
Arbitrary-execution grant (S)	161	100.0% (97.7–100.0)	83	0	83 (100%)
Agent description missing (Q)	158	99.4% (96.5–99.9)	23	1	22 (96%)
Skill without frontmatter (P)	306	100.0% (98.8–100.0)	78	0	78 (100%)
Skill description missing (P)	63	100.0% (94.3–100.0)	4	0	4 (100%)
Local settings committed (S) ^P	22	100.0% (85.1–100.0)	22	3	19 (86%)

References

- [1] AGENTS.md. 2026. AGENTS.md: An Open Format for Guiding Coding Agents. Agentic AI Foundation, The Linux Foundation. <https://agents.md>
- [2] C. Kevin, N. Raghavan, J.-F. Puget, R. Malani, M. Puvvadi, M. Abramovitch, M. Gupta, R. Akkiraju, S. Prabhu, Y. Dang, W. Luo, and S. H. Lee. 2026. Evaluating Skills, Not Just Agents: Agentic Continuous Evaluation of Skills. arXiv:2608.20614.
- [3] Y. Liu, W. Wang, R. Feng, Y. Zhang, G. Xu, G. Deng, Y. Li, and L. Zhang. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. arXiv:2601.10338.
- [4] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, Y. Zhang, and L. Y. Zhang. 2026. Malicious Agent Skills in the Wild: A Large-Scale Security Empirical Study. arXiv:2602.06547.
- [5] O. Yomtov. 2026. ClawHavoc: Malicious Skills on a Public Marketplace. Koi Security research blog.
- [6] National Vulnerability Database. 2025. CVE-2025-59536.
- [7] D. Schmotz, L. Beurer-Kellner, S. Abdelnabi, and M. Andriushchenko. 2026. SkillInject: Measuring Agent Vulnerability to Skill File Attacks. arXiv:2602.20156.
- [8] Z. Li, J. Wu, X. Ling, X. Cui, and T. Luo. 2026. Towards Secure Agent Skills: Architecture, Threat Taxonomy, and Security Analysis. arXiv:2604.02837.
- [9] T. Shen, Y. Feng, K. Zhu, X. Jia, Y. Liu, and L. Zhang. 2026. Sealing the Audit-Runtime Gap for LLM Skills. arXiv:2605.05274.
- [10] Z. Li, J. Cui, X. Liao, and L. Xing. 2026. Les Dissonances: Cross-Tool Harvesting and Polluting in Pool-of-Tools Empowered LLM Agents. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
- [11] F. Liu, Y. Zhang, J. Luo, J. Dai, T. Chen, L. Yuan, Z. Yu, Y. Shi, K. Li, and C. Zhou. 2025. Make Agent Defeat Agent: Automatic Detection of Taint-Style Vulnerabilities in LLM-based Agents. In *Proceedings of the 34th USENIX Security Symposium*. 3767–3786.
- [12] P. He, C. Li, B. Zhao, T. Du, and S. Ji. 2026. Automatic Red Teaming LLM-based Agents with Model Context Protocol Tools. *IEEE Transactions on Information Forensics and Security*. DOI 10.1109/TIFS.2026.3691201.
- [13] X. Hou, Y. Zhao, and H. Wang. 2025. On the (In)Security of LLM App Stores. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [14] U. Iqbal, T. Kohn, and F. Roesner. 2024. LLM Platform Security: Applying a Systematic Evaluation Framework to OpenAI’s ChatGPT Plugins. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society (AIES)*.
- [15] C. Yan, R. Ren, M. Zhang, et al. 2024. Exploring ChatGPT App Ecosystem: Distribution, Deployment and Security. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- [16] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*.

- [17] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS)* 37. 82895–82920.
- [18] S. Wang, Y. Zhao, X. Hou, and H. Wang. 2025. Large Language Model Supply Chain: A Research Agenda. *ACM Transactions on Software Engineering and Methodology*.
- [19] M. Chiari, M. De Pascalis, and M. Pradella. 2022. Static Analysis of Infrastructure as Code: A Survey. In *IEEE International Conference on Software Architecture Companion (ICSA-C)*. 218–225.
- [20] T. Sharma, M. Fragkoulis, and D. Spinellis. 2016. Does Your Configuration Code Smell? In *Proceedings of the International Conference on Mining Software Repositories (MSR)*.
- [21] A. Rahman, C. Parnin, and L. Williams. 2019. The Seven Sins: Security Smells in Infrastructure as Code Scripts. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- [22] H. V. F. dos Santos, V. Costa, J. E. Montandon, L. L. Silva, and M. T. Valente. 2026. Configuration Smells in AGENTS.md Files: Common Mistakes in Configuring Coding Agents. arXiv:2606.15828.
- [23] W. Chatlatanagulchai, H. Li, Y. Kashiwa, B. Reid, K. Thonglek, P. Leelapruete, A. Rungsawang, B. Manaskasemsak, B. Adams, A. E. Hassan, and H. Iida. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884.
- [24] T. Gloaguen, N. Mündler, M. Müller, V. Raychev, and M. Vechev. 2026. Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv:2602.11988.
- [25] J. L. Lulla, S. Mohsenimofidi, M. Galster, J. M. Zhang, S. Baltes, and C. Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv:2601.20404.
- [26] S. Park, W. Kim, R. Tan, J. Zhang, W.-S. Han, P. Gao, C. Park, Y. Yao, R. Fu, E. Nallipogu, Q. Lin, S. Rajmohan, and D. Zhang. 2026. AutoSaddler: Automatic Harness Optimization with Durable Updates from Agent Execution Traces. arXiv:2608.23041.
- [27] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge. 2013. Why Don't Software Developers Use Static Analysis Tools to Find Bugs? In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 672–681.
- [28] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Communications of the ACM* 53, 2, 66–75.
- [29] T. Kremenek, K. Ashcraft, J. Yang, and D. Engler. 2004. Correlation Exploitation in Error Ranking. In *Proceedings of the 12th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 83–93.
- [30] A. Austin and L. Williams. 2011. One Technique Is Not Enough: A Comparison of Vulnerability Discovery Techniques. In *Proceedings of the International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 97–106.
- [31] OWASP. 2025. OWASP Top 10 for Large Language Model Applications.
- [32] MITRE. 2025. MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems.